Principles & Practices Document: AI Model Results Analysis Platform (AMRAP)

Course: CS5013 PAI | Pre-Evaluation Deliverable
Name: Ferdeno | **Roll No:** EE26S062 **Repository:** github.com/FerdenoIITM/AI-Model-Results-Analysis-Platform

Project. AMRAP stores AI model evaluation results (raw JSON on disk, metadata and six metrics in PostgreSQL), lets users filter, sort and page through them, and answers natural-language questions with retrieval-augmented generation (RAG) over **pgvector** and a local **Ollama** LLM. Stack: Spring Boot 3 (Java 21, multi-module Maven), React 18 + Vite, PostgreSQL 16, Docker Compose, JWT role-based access (superadmin, admin, member). The project was built with an AI coding agent; the sections below map each practice to the course module it comes from.

1. Prompt Engineering (Module 2)

I treat a prompt as the entire input the model sees, so the job is to make it *dense*, not long: every clause should remove an ambiguity. The habits I use, in the lecture's terms, with evidence from `Prompt.md` (full samples in Appendix A):

Lecture principle	How I apply it	Evidence from my prompts
Five-part anatomy (role, task, context, constraints, examples)	Checked before sending; if three parts are missing, the answer will be a guess. Role is usually "a coding agent editing this repo".	"Audience: A coding agent that will edit this repository and must not invent a new RAG pipeline."
Specificity	Replace "make it work" with an exact target, the contract to keep, and a verifiable *Done when* line.	"Done when a follow-up pronoun uses the previous turns, and today / yesterday limit search to that local day."
Curated context	Name the exact files, methods and terminal line ranges instead of pasting the repo; tell the agent what already exists.	"In terminals/4.txt lines 98-105, why is the container restarting?" / "ask() embeds request.question() only..."
CO-STAR framework	Used once a prompt is a screenful. I added an Output slot and use R for Response constraints plus Done-when.	Superadmin-only submission and chat follow-up prompts (Appendix A.1).
Negative prompting	Explicit "Do NOT" lists, placed high, to stop unrelated refactors and scope creep.	"Do NOT: change the existing Chat UI design; modify the existing RAG pipeline; refactor unrelated code."
Few-shot examples	One concrete input/output pair to pin an unusual shape (query rewrite, JSON response layout).	"what are the results today" + "results of them" becomes "results ingested today".
Iterate with short follow-ups	When an answer is mostly right, fix one thing in a two-line follow-up instead of rewriting the whole prompt.	"Send the whole session, not the last 6 turns." then "Is there any trade-off?" (A.3)
Prompt chaining (plan, generate, critique, patch)	Big features start in plan mode: the agent asks questions, I approve the plan, then it implements, then tests critique it.	"Give me a plan for implementing this using RAG." Filtering prompt: "Give me an implementation plan in this order: Phase 1 Database..."
Verification habit (anti-hallucination)	Tell the agent not to assume APIs; then every change must compile, run and pass tests before I accept it.	"Do not assume this exact response structure... Inspect the existing API architecture."

Prompts inside the product. The same rules shape the chat feature in `ChatService.java`. The system prompt gives a role ("AMRAP's analytics assistant"), grounds the model ("answer strictly from the CONTEXT chunks... do not invent numbers") and asks it to cite model, version and iteration. Retrieved chunks are wrapped in delimiters labelled with `result_id`, model, version and date (RAG context injection). Follow-ups use a two-step **chain**: a rewrite prompt with a structured-output rule ("Return only that search sentence, with no quotes"), whose reply is parsed defensively, then the answer prompt. Temperature is `0.2`, the lecture's setting for routine logic, and each history turn is capped at 2,000 characters (context pruning). Known gap and next step: the user question is not yet inside tags with a sandwich reminder, which is the lecture's defence against prompt injection.

2. Debugging (Module 4)

The rule I follow is **explain, then fix**: the agent must state the cause before it patches anything. My loop:

```
Reproduce -> narrow the context (smallest set of files / log lines) -> explain and rank likely causes ->
confirm with one experiment -> smallest patch, reviewed as a diff -> verify, then add a regression test
```

- **Context dial:** I start narrow (the exact terminal lines or failing test) and widen only if my guess about where the bug is was wrong.
- **Rubber-ducking with the model:** I restate my understanding and ask the model to confirm it ("So you are saying Postgres is on the host and the API is in Docker... Is that correct?").
- **Minimal repro:** a failing test is the repro; once it fails for the right reason, the fix is usually small.
- **Knowing when AI is the wrong tool:** the model cannot see my machine or time, so environment- and timing-dependent bugs were confirmed by my own evidence (logs, repeated runs).

Issue	Evidence and diagnosis	Fix and verification
API container restart loop (environment-dependent)	Logs showed the API could not reach PostgreSQL. Explanation first: Postgres ran on the host, the API in Docker, in different network namespaces, with Postgres listening only on localhost.	API points at `host.docker.internal` (`host-gateway`); host Postgres accepts Docker connections. Re-read logs until healthy.
Wrong status codes found by new tests	403 instead of 401 without a token; 500 for a malformed UUID or unknown route; nulls first in descending sort; 500 on concurrent duplicate email.	JSON 401 entry point, exception-to-status mapping, NULLS LAST + id tie-breaker, unique violation to 409. Each held by a test.
Flaky E2E pagination (time-dependent)	Page 2 "overlapped" page 1: the page re-rendered before loading started, and iteration IDs are second-resolution timestamps.	Wait for the real `GET /api/results` response; key rows by model+version. 3 repeated runs, 42/42 stable.

I check that a test really guards a fix by breaking the code on purpose (disabling the Clear Filters reset made its test fail).

3. Version Control with Git: Code and Prompts (Module 3)

- **Branches:** all development happens on `dev`; `main` is kept for a stable, evaluated version.
- **AI-drafted, human-checked commits:** I use the IDE's commit-message generator on the staged diff, then review it: the model sees *what* changed, so I correct the intent and add the *why* by hand. One commit maps to one prompt or feature, so the log reads as a changelog (e.g. `9f4541a` "require X-Ingest-Key or superadmin JWT for submissions", `fb68e66` "history tracking for follow-up questions"). If a generated message comes out vague, I split the commit. Tests run before committing AI-written changes.
- **Prompts are versioned like code:** `Prompt.md` records every prompt given to the agent and is committed alongside the source, so `git log -p Prompt.md` shows how prompts evolved next to the code they produced. The product's own prompts (`SYSTEM_PROMPT`, `REWRITE_PROMPT`) live in Java source, so any change to them is a reviewed diff.
- **Secrets stay out of git:** `.env` is ignored; `.env.example` documents every variable with placeholders; build output, `node_modules`, coverage and test reports are ignored too.

4. Testing, Refactoring and Documentation (Module 4)

Testing. `make check` runs six suites and prints a pass/fail table with coverage, exiting non-zero on any failure (`make test` starts the isolated test stack). Following the lecture's **edge-case enumeration**, tests are grouped by category: boundary values (0, 100, 100.0001), null/empty/wrong-type input, resource limits (10,000-row datasets), and concurrency (simultaneous creates). **Adversarial inputs** (malformed UUIDs, invalid filter operators, missing or forbidden tokens) must produce clean 4xx errors. The OpenAPI file acts as the **contract**: API black-box tests assert status codes and response shapes against it. Only the LLM is stubbed.

Suite	Tools	Tests	What it verifies
Frontend components	Vitest, RTL, MSW	198	Loading/empty/error states, filters, Clear Filters, sorting, pagination, modals.
Backend unit	JUnit 5, Mockito	203	Filter parsing edge cases, services, JWT and auth filters.
Backend integration	Testcontainers	117	Repository queries, every endpoint, security matrix, DB failure, concurrent writes.
Database / API / E2E	psql, fetch, Playwright	404	Constraints, trigger, 10k-row paging; status codes, CORS, chat 502; full user flows.
Total (all passing)		**922**	**Line coverage: 99.2% frontend, 96.9% backend.**

Refactoring. Refactoring changes shape, not behaviour, so the tests decide: I rerun the suite after every AI refactor to catch *silent behaviour change*. Refactor prompts follow the lecture shape: the code, the one move wanted, constraints ("keep the public signature", "do not refactor unrelated code") and the output format. Examples: error handling extracted into one exception handler; null ordering moved into the repository query; unused `failed_login_attempts`/`locked_until` columns removed with their code; result storage simplified to `<uuid>.json`. Maven modules (domain, persistence, application, web, security) and React feature folders keep concerns separate.

Documentation. Docs change in the same commit as the code: `README.md`, `docs/architecture.md`, `docs/data-model.md`, `docs/api.md` and `packages/openapi/openapi.yaml`, the source of truth for the API. Prompts that change behaviour ask for these updates explicitly ("Update the ChatRequest schema in openapi.yaml and the POST /api/chat note in docs/api.md").

Appendix A: Sample Prompts (abridged excerpts from Prompt.md and ChatService.java)

A.1 CO-STAR specification with a few-shot example (chat follow-ups)

```
C - Context: You are changing the existing AMRAP repo. Inspect the real code before editing. ChatService.ask()
embeds request.question() only, then prompts the chat model with one SystemMessage and one UserMessage. Prior
turns are not sent. [...]
O - Objective: Make follow-up questions and the words "today" / "yesterday" work, with the smallest change.
[...] Example: history question "what are the results today" plus new question "just show me the results of
them" becomes a search like "results ingested today".
O - Output: ChatPage sends history from its existing messages state. A short summary of files changed and how to
verify.
S - Style: Precise and local. Extend ChatRequest, ChatRequestBody, and ChatService.ask. Do not add a new chat
endpoint.
T - Tone: Direct and implementation-focused.   A - Audience: A coding agent that must not invent a new RAG
pipeline.
R - Response constraints: Do not: change the embedding model, chat model or vector size; store chat history in
Postgres; put secrets in git.
Done when a follow-up pronoun uses the previous turns, and "today" / "yesterday" limit search to that local day.
```

Principles: CO-STAR, curated context, few-shot example, negative prompting, verifiable Done-when.

A.2 Debugging: explain first, then rubber-duck the diagnosis (API restart loop)

```
In terminals/4.txt lines 98-105, why is the container restarting, and where are the log files I can read to find
out why?
First explain the problem - do not jump to the fix. State the issue clearly.
So you are saying Postgres is running on the host and the API is running inside Docker - they are in different
network namespaces and cannot talk to each other by default. Is that correct?
What is the fix for this? How do I let the API container reach host Postgres?
```

Principles: narrow context (exact line range), explain then fix, rubber-ducking to confirm the hypothesis before patching.

A.3 Short iterative follow-ups instead of a rewrite

```
How many previous messages are being sent to the model as context - the whole conversation, or only the last 6
turns?
I want the whole conversation of the current session sent as context, not just the last 6 turns.
Is there any trade-off to sending the entire session as context instead of only the last 6 turns?
```

Principles: iterate one change at a time; ask for trade-offs (context-window cost) before accepting.

A.4 Edge-case enumeration for the test suite

```
Implement a complete automated test suite covering Frontend, Backend, API, and Database. Do not change
application functionality just to make tests pass.
Do not test only happy paths. For every important feature include: valid, invalid, empty and null input;
boundary values; incorrect data types; duplicate data; large datasets; no matching results; API, database and
network failures; unauthorized/forbidden access; concurrent operations.
Do not create fake tests that only mock everything. make test runs the complete test suite and returns:
passed/failed count; coverage; non-zero exit code when any required test fails.
```

Principles: edge cases by category (Module 4), negative prompting against fake tests, one verifiable entry point.

A.5 Prompts inside the product (ChatService.java, temperature 0.2)

```
SYSTEM: You are AMRAP's analytics assistant. Answer the user's question strictly from the CONTEXT chunks below.
Each chunk [...] is labelled with its result id, model name, version, and iteration. If the context does not
contain enough information to answer, say so plainly - do not invent numbers. When you cite a value, mention the
model, version, and iteration it came from.
REWRITE: Rewrite the latest user question into one standalone search sentence. Resolve pronouns such as them,
those, and it using the conversation. Return only that search sentence, with no quotes.
```

Principles: role, grounding against hallucination, delimited RAG context, prompt chaining, structured output, temperature matched to the task.